Octyne: A Distributed Server Process Manager with Per-Application Execution Modes and Shared Sessions

Ibrahim Ansari
Dept. of Computer Engineering and Technology
MIT World Peace University
Pune, India
TODO-email

Chayan Raj Bharati
Dept. of Computer Engineering and Technology
MIT World Peace University
Pune, India
TODO-email

Omkar Darekar
Dept. of Computer Engineering and Technology
MIT World Peace University
Pune, India
TODO-email

Pratvina Vinayak Talele
Dept. of Computer Engineering and Technology
MIT World Peace University
Pune, India
TODO-email

***Abstract*— Small self-hosted deployments of two or three machines fall between two kinds of tool. Single-host supervisors such as systemd and Supervisor manage processes well but expose no remote interface, so routine work goes through SSH. Panel-based managers such as Pterodactyl offer a browser console and file access but require a central panel and database, which becomes the only point of control. This paper presents Octyne, a server process manager written in Go that runs as one binary per node. Octyne exposes one HTTP and WebSocket API for lifecycle control, a live console and file management; shares login tokens across nodes through Redis so that one login works everywhere; and keeps every node in charge of its own applications when the dashboard is unavailable. Its *ProcessRenderer* backend lets the operator choose, per application, whether it runs in-process (*integrated*), as a systemd transient unit that survives a restart of the manager, or as a Docker or Podman container with CPU and memory limits, a persistent volume and a restart policy. We describe the design and implementation and evaluate Octyne on a three-node cluster. TODO: One or two sentences with the headline numbers: CPU/memory overhead of systemd and docker modes relative to integrated, start-up time per mode, and recovery time after an Octyne restart.**

***Index Terms*—process management, containers, systemd, self-hosting, distributed systems, Go**

I. Introduction

Operators of small self-hosted deployments run a handful of long-lived services, a game server, a Node.js API, a static site, on two or three machines. The tools available to them split into two groups. Single-host supervisors (systemd [1], Supervisor, runit) provide restart policies and, in systemd's case, cgroup limits, but offer no HTTP API, web console or file access; day-to-day work needs a shell on each machine. Panel-based managers (Pterodactyl, Pelican, PufferPanel) provide a browser console and file manager, but depend on a central panel and database through which every administrative action passes; if the panel is down, processes keep running but nobody can manage them. Cluster orchestrators (Borg, Kubernetes [2], [3]) solve isolation and limits at a scale and operational cost that does not fit a three-node cluster, and even the lightweight Kubernetes distributions spend hundreds of megabytes and a CPU core on their control plane [4].

This paper presents Octyne, a process manager that sits between these groups. Octyne is one Go binary per node. It exposes an HTTP and WebSocket API for start, stop and kill, a browser console, and a file manager scoped to each application's directory. Login tokens are stored in a shared Redis instance, so a token issued by any node is valid on every node and a single dashboard can show the whole cluster. There is no central database and no scheduler: each node manages its own applications and keeps doing so if the dashboard disappears. Execution is pluggable through a *ProcessRenderer* interface with three implementations chosen per application: *integrated* (a child process of the manager), *systemd* (a transient unit that outlives a restart of Octyne), and *docker* (a Docker or Podman container with an image, CPU and memory limits, a persistent volume and a restart policy).

The contributions are:

- The design of a single-binary process manager for small clusters that combines remote management, shared sessions and self-managing nodes without a central service beyond a token store (Section III).
- A per-application execution backend with a common lifecycle, console and statistics interface across in-process, OS-supervised and containerised execution (Section IV).
- An implementation in Go and an evaluation on a three-node cluster measuring the overhead of each execution mode, start-up time, and recovery after a manager restart and after node and dashboard failures (Section VI). **TODO:** Adjust the list if the final evaluation differs.

The rest of the paper is organised as follows. Section II reviews related work. Section III describes Octyne's architecture, and Section IV the ProcessRenderer. Section V covers the implementation. Section VI reports the evaluation, Section VII discusses limitations, and Section VIII concludes.

II. Related Work

Remote management over HTTP. Fielding's REST constraints [5] established HTTP as the standard way to manage a remote resource without a shell. The managers that run most self-hosted services do not expose such an interface. systemd [1] offers unit files, dependency ordering and cgroup-based resource control on one host, but no HTTP API or browser console.

Restart as recovery. Candea and Fox argue that restart should be the normal recovery path and that components must be crash-safe [6]. systemd and PM2 implement this per host. Nothing in that group gives several hosts a shared login or a common view.

Live consoles. RFC 6455 [7] defines the WebSocket channel, and measurements show it beats HTTP polling on latency and overhead for live streams [8]. The protocol has no login or per-connection authorisation model; Octyne adds one with a one-time ticket.

Self-managing nodes. Brewer's retrospective on CAP recommends keeping work next to the node that performs it, so the node keeps running through a partition [9]. Panel-based tools route all administration through a central panel, which is the opposite choice.

Containers and isolation. Soltesz et al. established that OS-level containers give near-native performance and higher density than hypervisors [10]. Felter et al. measured Docker overhead as near zero for CPU and memory with costs only in some I/O paths [11], and Espe et al. updated the picture for runc, gVisor and Kata [12]. Casalicchio and Iannucci survey runtimes, orchestrators and security [13]. Borg [2] and its successors [3] supply per-task limits and health checks through a central scheduler; Koziolek and Eskandani quantify what even the lightweight Kubernetes distributions cost on constrained hardware [4]. None of these addresses a single-binary manager that offers containers as an option per application.

Panel-based managers. **TODO:** One paragraph on Pterodactyl, Pelican, PufferPanel, MCSManager and Crafty Controller, with the feature matrix from the comparative analysis as a table (console, files, HTTP API, web UI, multi-node, Docker, general apps, local CLI). Cite project documentation as the source.

III. Design

A. Goals

Octyne is designed around five goals. (1) One binary per node, with no service required beyond a token store. (2) All day-to-day operations, lifecycle, console and files, available over HTTP and WebSocket, so that no SSH session is needed. (3) One login for the whole cluster. (4) Every node keeps managing its own applications when the dashboard or any other node is unavailable. (5) The execution mode is chosen per application, and the modes that run outside the manager's process tree keep the application running across a restart of the manager.

B. Architecture

Figure 1 shows the architecture. Each node runs one Octyne process. An operator works through a web dashboard, a single Next.js application configured with the list of nodes, or through `octynectl`, a command-line tool that talks to the local node over a Unix socket without a login. The only shared service is a Redis instance holding login tokens.

TODO: Insert the architecture figure: operator and web dashboard at the top, two node boxes each containing Octyne (Connector, RedisAuthenticator and ticket store, Process ×N, ProcessRenderer, the three modes) above the three runtimes, Redis between the nodes, octynectl on node 1. Export from the presentation deck as PDF or SVG.

Fig. 1. Octyne architecture with two nodes. Each node runs the same binary; the dashboard reaches every node with one token held in the shared Redis.

Inside a node, four components cooperate.

Configuration. A `Config` read from `config.json` holds the listen port, an optional Unix socket, HTTPS settings, a `RedisConfig`, and a map of `ServerConfig` entries, one per managed application. A `ServerConfig` names the application's `Directory` and `Command`, whether it is `Enabled`, and the execution fields described in Section IV: `Renderer`, `Image`, `CPULimit`, `MemLimit` and `RestartPolicy`.

Connector. The `Connector` registers the HTTP routes and holds a map of `ExposedProcess` entries and a map of `Tickets`. The routes are `/login` and `/logout`; `/servers` for the status of every application; `/server/{id}` for start, stop and kill and for CPU, memory and uptime; `/ott` for a one-time console ticket; `/server/{id}/console` for the WebSocket console; and `/server/{id}/files`, `/file`, `/folder`, `/compress` and `/decompress` for file management under the application's directory.

Authentication. Every route except `/login` passes through an `Authenticator` interface with `Login`, `Logout`, `Validate` and `HasPerm`. `MemoryAuthenticator` keeps users and tokens in memory for a single node. `RedisAuthenticator` stores tokens in the shared Redis with a TTL, so that a token issued by one node validates on every node; this is what lets one dashboard login cover the cluster. For the console, a client first calls `/ott` with its token and receives a short-lived `Ticket` bound to its IP address; it then opens the WebSocket with the ticket in the URL. The `Connector` validates and deletes the ticket, adds the client's channel to `ExposedProcess.Clients`, sends the existing console buffer, and fans new output out to every attached client. Input from any client is written to the application.

Process. A `Process` holds the state of one application: `Online`, `Crashes`, `Uptime`, and the output and input pipes. It exposes

`StartProcess`, `StopProcess`, `KillProcess`, `SendCommand` and `MonitorProcess`. `MonitorProcess` polls CPU and memory once a second and serves them on `GET /server/{id}`; it also detects an unexpected exit, increments `Crashes` and applies the restart policy. `ExposedProcess` wraps `Process` with the console buffer and the set of attached WebSocket clients.

C. Multi-node operation

Nothing in Octyne is cluster-aware beyond the token store, and that is deliberate. The dashboard reaches each node directly with the same token. Its node list shows reachability (whether `GET /servers` answers), CPU, total memory and the status of every application per node, so the operator can see where there is room before registering an application. There is no scheduler: the operator picks the node. If the dashboard is down, every node keeps running its applications and `octynectl` keeps working on each machine. If one node is unreachable, the others are unaffected and the shared login continues to work on them.

IV. ProcessRenderer

A. Interface

A `ProcessRenderer` sits between `Process` and the operating system:

```
type Handle struct { ID string; PID int; Kind
string }

type ProcessRenderer interface {
    Start(cfg ServerConfig) (Handle, error)
    Stop(h Handle) error
    Kill(h Handle) error
    Attach(h Handle) (stdin io.WriteCloser,
                      stdout io.ReadCloser, err
error)
    Stats(h Handle) (Stats, error)
    Adopt(name string) (Handle, bool)
}
```

`Process.StartProcess` reads `ServerConfig.Renderer`, selects the matching renderer, calls `Start`, and attaches the returned streams to the output pipe so that the console buffer and the WebSocket fan-out behave the same in every mode. `MonitorProcess` calls `Stats`. On start-up, Octyne calls `Adopt` for every enabled application; a renderer that finds a running unit or container returns its handle and the application is reported as online without being restarted.

B. The three renderers

IntegratedRenderer runs the command with `exec.Command` as a child of the Octyne process, with the working directory set to `Directory`. It is the default and the lightest mode. The application stops when Octyne stops; `Adopt` always returns false.

SystemdRenderer creates a transient unit through the systemd D-Bus API (equivalent to `systemd-run`), named from the application, with `WorkingDirectory` set to `Directory`, `CPUQuota` and `MemoryMax` from the limits, and `Restart=` from the restart policy. Output is read from the journal or from a pseudo-terminal the unit is attached to. **TODO:** State which of the two you chose and why. The unit outlives Octyne; `Adopt` queries the unit's `ActiveState` and returns its main PID.

DockerRenderer talks to the Docker Engine API or the Podman-compatible socket. `Start` pulls `Image` if absent, creates a container with the command, a bind mount of `Directory` as the persistent volume, CPU and memory limits, and the restart policy, then starts and attaches to it. Stop sends the configured stop signal with a grace period; Kill removes the container. `Adopt` looks up a container labelled with the application name.

Table 1 summarises what each mode provides.

TABLE 1
Properties of the three execution modes.

Property	integrated	systemd	docker
Survives Octyne restart	no	yes	yes
CPU and memory limits	no	cgroup	cgroup
Filesystem isolation	no	no	yes
Reproducible image	no	no	yes
External dependency	none	systemd	Docker or Podman

V. Implementation

TODO: Describe the implementation: Go version, size of each package (`connector`, `auth`, `process`, `renderer`, `system`, `config`), the systemd D-Bus and Docker SDK libraries used, how the Podman socket is detected, how `Adopt` runs at start-up, the dashboard (Next.js version, pages) and `octynectl`. Include the licence and repository URL as a footnote.

TABLE 2
`ServerConfig` fields. **TODO:** Confirm names, types and defaults against the code.

Field	Type	Default	Used by
`Directory`	string	–	all
`Command`	string	–	all
`Enabled`	bool	true	all
`Renderer`	string	`integrated`	all
`Image`	string	–	docker
`CPULimit`	float	0 (no limit)	systemd, docker
`MemLimit`	int64 (bytes)	0 (no limit)	systemd, docker
`RestartPolicy`	string	`on-failure`	all

VI. Evaluation

A. Setup

TODO: Three Ubuntu 24.04 nodes: hardware (CPU model, cores, RAM, disk), physical or virtual, network between them. Redis 7 version and where it runs. Docker or Podman version. Go version. The three workloads: a Java jar (which one), a Node.js service, a static site served by what. Number of repetitions per measurement and how variance is reported.

B. Overhead per mode

We measure the CPU and memory of each workload under the three renderers, with the integrated mode as the baseline, following the method of Felter et al. [11] and Espe et al. [12]. **TODO:** Describe the load applied to each workload and the sampling interval.

TABLE 3
Steady-state CPU (percent of one core) and resident memory (MB) per workload and execution mode, mean of **TODO:** N runs.

Workload	int. CPU	sysd CPU	dock CPU	int. mem	sysd mem	dock mem
Java jar	**TODO:** x	**TODO:** x	**TODO:** x	**TODO:** x	**TODO:** x	**TODO:** x
Node.js service	**TODO:** x	**TODO:** x	**TODO:** x	**TODO:** x	**TODO:** x	**TODO:** x
Static site	**TODO:** x	**TODO:** x	**TODO:** x	**TODO:** x	**TODO:** x	**TODO:** x

TODO: One paragraph reading the table. The expected result is that steady-state figures are close across modes; say so if that is what you find, and explain why (a cgroup and a few namespaces cost little once the process is running).

C. Start-up time

Start-up time is where the three modes differ by construction: integrated mode forks a process, systemd mode makes a D-Bus round trip and a unit activation, and docker mode checks the image and sets up namespaces and mounts. We measure the time from `POST /server/{id}` with action `start` to the first line of application output on the console WebSocket. **TODO:** State whether the image was pre-pulled for docker mode, and report cold and warm cases separately if you measured both.

TABLE 4
Start-up time per workload and execution mode, median of **TODO:** N runs with **TODO:** interquartile range or standard deviation.

Workload	integrated (ms)	systemd (ms)	docker (ms)
Java jar	**TODO:** x	**TODO:** x	**TODO:** x
Node.js service	**TODO:** x	**TODO:** x	**TODO:** x
Static site	**TODO:** x	**TODO:** x	**TODO:** x

D. Recovery after an Octyne restart

For each mode we start the three workloads, restart the Octyne binary (`systemctl restart octyne`), and record whether each application kept running, whether Octyne re-adopted it, and how long the console was unavailable.

TABLE 5
Behaviour across an Octyne restart.

Mode	App kept running	Re-adopted	Console gap (s)
integrated	no (by design)	–	**TODO:** x
systemd	**TODO:** yes/no	**TODO:** yes/no	**TODO:** x
docker	**TODO:** yes/no	**TODO:** yes/no	**TODO:** x

E. Recovery after node and dashboard failure

TODO: Describe the two fault tests: (a) stop the web dashboard and show that `octynectl` and direct API calls to each node still work and that applications keep running; (b) disconnect one node from the network and show that the other node and its applications are unaffected, the dashboard marks the node unreachable, and the shared login still works on the remaining node. Report the time until the dashboard shows the node as unreachable.

F. Crash recovery

TODO: Optionally: kill an application process (`kill -9`) under each mode and report the time until it is back online and whether `Crashes` increments. This exercises the restart policy in each renderer.

VII. Discussion

What Octyne does not do. There is no scheduler: the operator picks the node for each application, informed by the dashboard's per-node figures. There is no central database; Redis holds login tokens only. There is no orchestration of multi-container applications and no application catalogue. These are deliberate exclusions that keep Octyne a single binary per node. **TODO:** Add anything the evaluation showed to be a limitation.

Threats to validity. **TODO:** Small cluster, one hardware configuration, three workloads, one container runtime per node, measurements taken by the authors. Say how each threat was mitigated or why it is acceptable for the claim being made.

Security. **TODO:** Note that the docker renderer talks to the Docker socket, which is root-equivalent, and what you did about it (rootless Podman, a dedicated socket, group membership). Note which user systemd transient units run as. Note that file endpoints are confined to the application directory and how path traversal is prevented.

VIII. Conclusion

Octyne gives a small self-hosted cluster a shared login, a live console, a file manager and per-application isolation from one Go binary per node, with Redis as the only shared service and no central panel in the control path. Its ProcessRenderer lets each application run in-process, under systemd or in a container, and the last two keep running across a restart of the manager. On a three-node cluster, **TODO:** one sentence with the main numbers. **TODO:** One sentence on future work, for example rootless Podman as the default or a fourth renderer.

Availability. **TODO:** Repository URL, licence (Apache-2.0), commit hash used for the evaluation, and where the benchmark scripts live.

IX. Acknowledgment

TODO: Thank the guide and anyone who provided hardware. Delete the section if not needed.

IX. Bibliography

[1] Z. Jędrzejewski-Szmek and J. B. Guðmundsson, "systemd service management," *USENIX ;login:*, vol. 41, no. 2, 2016.

[2] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with Borg," in *Proc. 10th European Conf. on Computer Systems (EuroSys)*, ACM, 2015.

[3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *ACM Queue*, vol. 14, no. 1, 2016.

[4] H. Koziolek and N. Eskandani, "Lightweight Kubernetes Distributions: A Performance Comparison of MicroK8s, k3s, k0s, and Microshift," in *Proc. ACM/SPEC Int. Conf. on Performance Engineering (ICPE)*, 2023.

[5] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Doctoral dissertation, 2000.

[6] G. Candea and A. Fox, "Crash-Only Software," in *Proc. 9th Workshop on Hot Topics in Operating Systems (HotOS IX)*, USENIX, 2003.

[7] I. Fette and A. Melnikov, "The WebSocket Protocol," technical report RFC 6455, 2011.

[8] V. Pimentel and B. G. Nickerson, "Communicating and Displaying Real-Time Data with WebSocket," *IEEE Internet Computing*, vol. 16, no. 4, 2012.

[9] E. Brewer, “CAP Twelve Years Later: How the ``Rules" Have Changed,” *IEEE Computer*, vol. 45, no. 2, 2012.

[10] S. Soltesz, H. Pötzl, M. E. Fiuczynski, A. Bavier, and L. Peterson, “Container-based Operating System Virtualization: A Scalable, High-performance Alternative to Hypervisors,” in *Proc. 2nd European Conf. on Computer Systems (EuroSys)*, ACM, 2007.

[11] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, “An Updated Performance Comparison of Virtual Machines and Linux Containers,” in *Proc. IEEE Int. Symp. on Performance Analysis of Systems and Software (ISPASS)*, 2015.

[12] L. Espe, A. Jindal, V. Podolskiy, and M. Gerndt, “Performance Evaluation of Container Runtimes,” in *Proc. 10th Int. Conf. on Cloud Computing and Services Science (CLOSER)*, SciTePress, 2020.

[13] E. Casalicchio and S. Iannucci, “The state-of-the-art in container technologies: Application, orchestration and security,” *Concurrency and Computation: Practice and Experience*, vol. 32, no. 17, 2020.